

Querying Database

Simple Query Usage

In order to execute a query on the database, `Database.RunQuery` can be used. The below example shows a query with a provided input:

```
var result = Database.RunQuery('SELECT ID, LoginName as "LoginName" FROM EMPLOYEE WHERE LoginName = @LoginName', { LoginName : loginName });
```

Query returns a `Database.SqlResult` object which contains query results.

```
//Query results are stored in results.Rows object which is a table.

DebugConsole.LogDebug("Found employee ID={0}, Login={1}", result.Rows[0].ID, result.Rows[0].LoginName);

//As result.Rows is a list of returned rows from the database, it is possible to execute an action for every row using forEach:

result.Rows.forEach(r => {
    DebugConsole.LogDebug("Found employee ID={0}, Login={1}", r.ID, r.LoginName);
});
```

Defining Query Parameters Data Type

The data type of query parameters is determined in runtime. It is possible that the determined data type will be invalid, especially when running a query on the Oracle database, or running a query on data type of date. In such cases the data type should be provided in the script.

An example below shows a script that generates a query and provides data types for query parameters:

```
var query = Database.Query.Create("SELECT ID FROM OPERATION WHERE OperationCode = @OperationCode AND CreatedOn >= @CreatedOn",
    {
        OperationCode : operationCode,
        CreatedOn: createdOn
    });

//Defining query parameter types:

query.DefineParameter("CreatedOn", Database.ParameterType.DateTime);
```

Using Oracle database

Running a query on the Oracle database returns column names written in upper-case, unless an alias for the column is provided. Depending on the query, the values from `SqlResult` can be accessed either via the capitalized column name, or the provided alias.

When the column alias is not provided in the query, the returned values can be accessed by using the column name has to be in upper-case:

```
var result = Database.RunQuery('SELECT LoginName FROM EMPLOYEE WHERE LoginName = @LoginName', { LoginName : loginName });

var ln = result.Rows[0].LOGINNAME;
```

When the column alias is provided in the query (e.g., in CamelCase as in the example below), it is possible to access returned values only with the use of the same alias:

```
var result = Database.RunQuery('SELECT LoginName as "LoginName" FROM EMPLOYEE WHERE LoginName = @LoginName', { LoginName : loginName });

var ln = result.Rows[0].LoginName;
```

Preparing Script to Work on SQL and Oracle databases (QueryRepository Usage)

In order to use the same script for both databases (SQL and Oracle), provide an alias for column names or return them in upper-case notation as SQL database behaves differently than Oracle.

If a query on the Oracle database is different than on the SQL server, use QueryRepository to store named queries and use these names to create a query object and execute the query.

Sample Usage

Query repositories initialization can be done in a separate class and just called in this one. Providing data type for parameters is optional.

```
//Registering a query in the repository (queries for SQL and Oracle database can be different):

Database.QueryRepository.RegisterQuery("MyQuery",
    "SELECT OperationCode, OperationRevision FROM OPERATION WHERE
IsDefaultRevision = @DefaultRevision AND OperationCode = @Code",
    'SELECT OperationCode AS "OperationCode", OperationRevision as
"OperationRevision" FROM OPERATION WHERE IsDefaultRevision = @DefaultRevision AND
OperationCode = @Code',
    [
        {Name : "DefaultRevision", DataType : Database.ParameterType.Boolean},
        {Name : "Code", DataType : Database.ParameterType.String}
    ]);

//Using a query from the repository by providing its name:

var query = Database.Query.CreateByName("MyQuery", {
    DefaultRevision : true,
    Code : "PortalExecute"
});

//Executing the query:

var result = query.Run();
```